

3. 第 3 堂：pandas 選取與摘要 (180 分鐘)

3.1 學習目標

- 建立「先 Profiling 再分析」的標準習慣。
- 熟練使用 `head`、`info`、`describe`、`value_counts` 做資料健檢。
- 掌握缺失值處理策略的判斷依據。
- 能進行資料型別轉換與日期/字串處理。

3.2 前情提要

上一堂學習使用 pandas 讀取與篩選資料。但在開始做任何分析之前，有一件事比篩選更重要：**你真的了解手上的資料嗎？**

為什麼這很重要：實務上，分析師最常犯的錯誤不是「用錯方法」，而是「對資料有錯誤的假設」。例如以為某欄位沒有缺失值，結果發現 40% 是空的；以為金額欄位是數值，結果是字串。Profiling 能讓你在犯錯前就發現問題。

3.3 標準 Profiling 四步驟

每次拿到一份新資料，執行以下四個步驟：

步驟 1：`df.head()` — 看長相

```
sessions = pd.read_csv("data/raw/sessions.csv")
print(sessions.head())
# 先看前 5 筆，建立對欄位的直覺感受
# 問自己：每一欄是什麼意思？值看起來合理嗎？
```

步驟 2：`df.info()` — 看結構

```
print(sessions.info())
# 輸出包含：
# - 每欄的名稱
# - Non-Null Count ( 非空值數量 ) → 反推缺失數量
# - Dtype ( 資料型別 ) → 確認是否需要轉型
# - 記憶體用量
```

預期輸出範例：

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 70000 entries, 0 to 69999
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   session_id            70000 non-null  int64
```

1	customer_id	70000	non-null	int64	
2	session_start	70000	non-null	object	← 應轉為 datetime
3	device	70000	non-null	object	
4	traffic_source	70000	non-null	object	
5	campaign	70000	non-null	object	
6	experiment_group	70000	non-null	object	

步驟 3：df.describe() — 看數值分布

```
print(sessions.describe())
# 只對數值欄位有效
# 觀察 min/max 是否異常 (例如負數金額)
# 觀察 mean 與 50% (中位數) 差距 → 判斷是否偏態
```

```
# 若要包含非數值欄位
print(sessions.describe(include="all"))
# 會額外顯示 unique (不重複值數量)、top (最常出現值)、freq (出現次數)
```

理解常見的資料數值分布名詞：

- **偏態 (Skewness)**：平均數與中位數差距大，且分布不對稱。
- **離群值 (Outliers)**：極端值遠離其他數據點，可能是錯誤或特殊情況。
- **集中趨勢 (Central Tendency)**：數據集中在某個值附近，平均數、中位數、眾數是衡量集中趨勢的指標。
- **平均數 (Mean)**：所有數值的總和除以數量，受極端值影響大。
- **中位數 (Median)**：將數值排序後的中間值，對偏態分布更穩健。
- **眾數 (Mode)**：出現頻率最高的值，適用於類別型資料。

步驟 4：value_counts() — 看類別分布

```
print(sessions["campaign"].value_counts())
# 輸出：
# none          38500
# new_user      10500
# retarget       7000
# spring_sale    8400
# double11       5600
# 觀察：是否有意外類別？分布是否極度不均？

print(sessions["device"].value_counts(normalize=True))
# 加上 normalize=True 會顯示百分比
# mobile        0.62
# desktop       0.33
# tablet        0.05
```

3.4 缺失值處理策略

缺失值並不是一律刪除或補值，而是你需要根據**業務情境**做判斷。

策略	適用情境	範例
刪除整列	缺失比例 < 5% 且缺失為隨機	少數訂單缺少 customer_id
補固定值	缺失有明確業務意義	campaign 欄空值 → 填 "none" (代表無活動)
補統計值	數值欄位、缺失比例適中	金額欄缺失 → 補中位數 (比平均數抗離群值)
補前/後值	時間序列資料	股價缺失 → 用前一天的值
標記為類別	缺失本身是訊號	退貨原因欄空白 → 可能代表「沒填」這件事本身有意義
保留不處理	後續模型可處理缺失	某些模型可直接處理 NaN

決策流程：

1. 先問：缺失的原因是什麼？ (隨機遺漏？系統問題？本來就沒有？)
2. 再問：缺失比例多高？ (< 5% 通常影響小，> 30% 要特別小心)
3. 最後問：填補方式會不會帶入偏誤？

3.5 資料型別轉換

日期時間轉換

```
# 字串 → datetime
sessions["session_start"] = pd.to_datetime(sessions["session_start"])

# 驗證轉換結果
print(sessions["session_start"].dtype) # datetime64[ns]
print("資料期間：",
      sessions["session_start"].min(), "至",
      sessions["session_start"].max())
```

數值型別確認

```
order_items = pd.read_csv("data/raw/order_items.csv")

# 確認金額欄位是數值
print(order_items["unit_price"].dtype) # int64 ✓
print(order_items["quantity"].dtype)  # int64 ✓

# 若不小心讀成字串，就需要手動轉換
# order_items["unit_price"] = pd.to_numeric(order_items["unit_price"],
# errors="coerce")
# errors="coerce" 會將無法轉換的值設為 NaN，而非報錯
```

字串清理

```
# .str 存取器提供豐富的字串操作
sessions["campaign_clean"] = sessions["campaign"].str.strip()    # 去頭尾空白
sessions["campaign_upper"] = sessions["campaign"].str.upper()    # 轉大寫
sessions["is_retargert"] = sessions["campaign"].str.contains("retarget") # 包含判斷

# 檢查是否有異常類別 ( 例如多餘的空白或拼字錯誤 )
print(sessions["campaign"].unique())
# ['none' 'spring_sale' 'double11' 'new_user' 'retarget']
```

3.6 範例操作與程式碼解析

```
uv run python lessons/lesson03.py
```

```
from common import ensure_packages, load_data

def lesson03(data) -> None:
    import pandas as pd

    print("Lesson 3: profiling, missing values, dtype, datetime")

    sessions = data["sessions"].copy()
    # .copy() 避免修改原始資料 ( 好習慣 )

    print(sessions.info())
    # 一次看完所有欄位的型別與缺失狀況

    print(sessions["campaign"].value_counts().head())
    # 看 campaign 欄的前 5 大類別
    # 確認分布是否合理、有無異常值

    sessions["session_start"] = pd.to_datetime(sessions["session_start"])
    # 將 session_start 從字串轉為 datetime

    print("Date range:",
          sessions["session_start"].min(), "to",
          sessions["session_start"].max())
    # 確認資料覆蓋的時間範圍
    # 如果最大日期遠超預期，可能是資料產生問題
```

3.7 建立欄位健康檢查表

以下是建立「欄位健康表」的標準樣板，這是分析師的基本功：

```
def column_health_report(df, table_name=""):
    """產出一張表的欄位健康報告。"""
    report = pd.DataFrame({
        "欄位": df.columns,
        "型別": df.dtypes.values,
        "非空數": df.notna().sum().values,
        "缺失數": df.isna().sum().values,
        "缺失率": (df.isna().sum() / len(df) * 100).round(2).values,
        "不重複值": df.nunique().values,
    })
    # 標註風險等級
    report["風險"] = "低"
    report.loc[report["缺失率"] > 10, "風險"] = "中"
    report.loc[report["缺失率"] > 30, "風險"] = "高"
    print(f"\n=== {table_name} 欄位健康報告 ===")
    print(report.to_string(index=False))
    return report

# 使用範例
health = column_health_report(sessions, "sessions")
```

3.8 章內實作

執行指令：

```
uv run python lessons/practice_ch03.py
```

任務 1：產出 `sessions` 的欄位健康表

- 使用上述樣板或自行設計，列出每個欄位的型別、缺失率、不重複值。

任務 2：標註高風險欄位

- 哪些欄位缺失率高？哪些欄位型別不正確？

任務 3：提出處理策略

- 針對每個有問題的欄位，寫出你的處理方案與理由。

3.9 章末練習

練習 1：若某欄位缺失 40%，你會刪除、補值或保留？

參考答案：不能一概而論，需考慮：

1. **該欄位對分析是否關鍵？** 如果是核心指標的分子/分母，40% 缺失會嚴重影響結果，需謹慎處理。
2. **缺失是隨機的嗎？** 如果缺失集中在特定族群（例如只有 mobile 用戶缺失），補值可能引入偏誤。

3. 可以用其他欄位推算嗎？例如 `campaign` 欄缺失，但可以從 `traffic_source` 推斷。

4. 最保守做法：標記為「缺失」類別，讓後續分析可以追蹤缺失的影響。

練習 2：以下 `describe()` 輸出有什麼異常？

	unit_price
count	39625.00
mean	1520.30
std	1280.50
min	-5.00 ← 注意這裡
25%	450.00
50%	1100.00
75%	2200.00
max	5000.00

參考答案：`min = -5.00`，單價出現負值，在正常商品定價中不合理。需要調查原因：是退款記錄？是折扣過頭？還是資料輸入錯誤？處置方式取決於原因，若是錯誤則標記為 `NaN` 或修正，若是退款則需與訂單狀態交叉確認。

3.10 本章小結

- 分析前必須做 **Profiling**，這不是可選項，是標準流程。
- 四步驟：`head` → `info` → `describe` → `value_counts`。
- 缺失值處理需回到商業問題判斷，不是無腦刪除或補值。
- 日期欄位一定要轉 `datetime`，金額欄位要確認是數值。

下堂預告：第 4 堂我們將學習 `groupby` 和 `pivot_table`，開始建立真正的 KPI 報表。